

From Informal System Requirements to Formal Software Specifications - An Experience Report

Anitha Murugesan
Honeywell Aerospace
Plymouth, MN, USA

anitha.murugesan@honeywell.com

Daniel Cofer
Squarespace
New York, NY, USA

daniel.austin.cofer@gmail.com

Michael Whalen
Amazon AWS
Minneapolis, MN, USA

mwwhalen@umn.edu

Mats Heimdahl
University of Minnesota
Minneapolis, MN, USA

heimdahl@umn.edu

Abstract—Formal methods have been enormously useful in verifying complex system requirements, especially in the safety critical domain. However, their success depends on precisely formalizing *what* needs to be verified and thoroughly understanding *how* it is verified. Unfortunately, there is a lack of awareness and guidance in using formal techniques effectively, that often makes their use difficult and the results of their application leading to overconfidence in the correctness of the fielded system in its intended environment. In this paper, we share some of the lessons learnt while using formal methods to specify and verify a complex infusion pump system. While the effort was successful and has led to hierarchically verified software for the pump, it was not without challenges that we believe are not adequately presented in the research literature. We discuss the challenges we faced with (a) precisely “flowing down” system-level requirements to low level components, (b) thoroughly understanding the technicalities of the formal tools to avoid faulty premises and misplaced confidence on the overall system and (c) rigorously mitigating risks with using diverse formal tools in hierarchical system analysis. In the sequel, we also discuss our approach to mitigating them. We believe that the lessons we learned in this effort serves informative for practitioners involved in similar efforts.

Index Terms—Formal Verification, Requirements Specification, Formal Methods

I. INTRODUCTION

In safety critical systems, formal methods (mathematical techniques) are often used to rigorously assure the correctness of systems. While advances in technology has improved the verification capability and efficiency of formal tools such as model checkers, they still rely on the skill level of the engineers to correctly formalize requirements and understand the technicalities of the tools. For instance, one could use a set of inconsistently formalized premises to prove requirements using formal tools and gain misplaced confidence about the successful but meaningless verification. While there are numerous success stories that illustrate the benefits of using formal methods [1,2] as well as discussions about the limitations of formal methods [3,4], we believe that there are not adequate discussions about the practical challenges encountered and techniques adopted to mitigate them in real applications. In our opinion, codifying the pitfalls will allow wider and better dissemination of formal methods in industrial use.

This work has been partially supported by NSF grants CNS-0931931 and CNS-1035715.

In prior work [5-7], we proposed scalable and efficient model based approaches to verify the formalized requirements of complex control systems, given their architecture (hierarchical decomposition into components) and formalized requirements. The approach involved hierarchically modeling, formalizing and verifying the requirements of the system and its components using multiple tools, each with slightly different formal representations and usage. While this approach was useful to verify hierarchically decomposed systems, when we used it to verify an substantially complex medical infusion pump system, we faced a number of practical challenges and pitfalls that had the potential to undermine the value of not just our approach but formal requirements analysis in general. To the best of our knowledge these challenges, that lie in the intersection of the requirements engineering and formal verification domains, have not been discussed in depth and addressed in any existing literature.

In this paper we discuss the challenges we faced while formally verifying requirements at multiple levels of hierarchy, applying tools and techniques “at scale” involving several developers over an extended period of time. We focus on three challenges:

a) Requirements Refinement: When systems are decomposed and requirements are allocated to its components, establishing the fidelity between the system and its component requirements is a challenge, especially when system requirements are informally expressed and component requirements are formally specified. While decomposing the infusion pump system, we derived formal software requirements from the natural language system requirements using an approach proposed by Miller et.al [8](also used in other research efforts such as [9-11]), but found that it led to incorrect compositions when we tried to establish system requirements.

b) Cursory Knowledge of Tools: In an hierarchical architectural proof approach, all proofs are predicated on “leaf level” components meeting their requirements. If these leaf-level requirements are incorrect (infeasible or inconsistent), then the proof is not well-founded. While formalizing the leaf level component requirements for the infusion pump, we unknowingly formalized some computations used within requirements in an unrealizable way, that was not detected by the tools and lead to misplaced confidence about the system.

c) **Matching Tool Boundaries:** In order to be able to reason at scale about complex systems, we found it necessary to use multiple reasoning tools at different levels of abstraction within the software architecture. This use necessitated translation of properties between multiple tools. However, the mere action of transcription of requirements between different notations of the tools (even through the tools shared the “same” semantics) induced several errors that lead to misplaced confidence in results.

The contribution of the paper is reporting the challenges and non-obvious nuances in using formal methods to verify requirements of complex systems that, we believe, engineers will likely encounter in practice. Further, we also elaborate our approach to addressing them. To the best of our knowledge, these issues and their potential mitigation have not been adequately discussed in the formal analysis literature. While we illustrate our lessons learned using the infusion pump, we believe many engineers working on similar applications, especially in the safety critical system domain, will face similar challenges and hence we hope that sharing our experience proves instructive.

The paper is organized as follows. In Section II we provide a brief overview of our previous effort that serves as a context for the rest of the paper. In Section III, we explain the challenges we encountered along with our approaches and recommendations to address them. Finally, we conclude the paper in Section IV.

II. BACKGROUND

In this section, we provide a brief overview of an infusion pump system, its modeling and verification activities [5,6] that is required to understand our work and the challenges described in the rest of the paper.

A. Infusion Pump System Overview

Infusion pumps are medical devices that are used to deliver controlled quantities of the drug into patient’s body. Unfortunately, these devices have been involved in many incidents that triggered the need to strengthen their development practices [12]. In that context, our aim is to identify and demonstrate a rigorous development approach of a generic infusion pump system using formal tools and techniques and release it publicly to be used as a reference for researchers, manufacturers and certification authorities.

We considered a Generic Patient Controlled Analgesia Infusion Pump (GPCA), a special type of infusion pump that allows patients to self-administer a controlled amount of additional drug. In order to analyze the problems associated with modern infusion pumps available in the market, we created a model of similar complexity to current generation commercial infusion pumps. We considered three types of infusion options for drug delivery for the GPCA - (a) *basal* infusion in which the drug is delivered at a constant (and usually low) rate for an extended period of time, (b) *intermittent bolus* infusion that delivers drug at a higher rate for a short duration at prescribed time intervals according to some therapy regimen, and (c) *patient bolus* that delivers additional drug in response

to a patient’s request for more medication. In addition, we also included advanced safety features in the device to detect hazardous anomalies/behaviors and notifications for clinicians and mitigations via inhibiting infusion and/or other device operations.

B. System Modeling and Verification

Our interest with the GPCA is to develop and demonstrate a rigorous end-to-end development approach, using model based techniques. To that end, we first captured a reasonably complete set of overall system requirements using documentation patterns [13] and standard templates [14] to simplify the task of formalizing requirements. During the modeling and verification the GPCA, to cope with its complexity, we followed a hierarchical decomposition approach that allows the modeling and verification to be systematically partitioned into smaller and more manageable tasks.

In this approach [5], the system is hierarchically decomposed into a set of interconnected components (its architecture) and each component is allocated its own set of requirements. We used the Architectural Analysis & Design Language (AADL) [15] to capture the architecture of the GPCA and an extension of the AADL language to capture formal requirements of the components (and system) until the leaf level. At the leaf level, in addition to the allocated requirements, the detailed behavior of each component is modeled. While the AADL notations allowed us to capture requirements of each component, it is not designed for constructing detailed behavioral models. Hence for the leaf level components we modeled their behavior using MathWorks Simulink/Stateflow notation and tool [16] - an industry standard tools for behavioral modeling. To verify the hierarchical architecture driven requirements decomposition, we used a compositional reasoning tool named AGREE [17]. Given the system architecture and the allocated requirements, AGREE hierarchically verifies if the system level guarantees holds as a logical consequence of its component requirements. To verify the leaf-level requirements with respect to their detailed behavioral model we used Simulink Design Verifier tool [16]. Since we used two different tools at the leaf level, we recaptured the requirements from AGREE notation to a notation supported by Simulink Design Verifier, namely Embedded Matlab [16]. Fortunately, the rewriting was straightforward since both the notations have the same underlying semantics (that of synchronous dataflow languages). This approach and tool chain allowed us to rigorously demonstrate a scalable end-to-end system verification.

While our approach scalably verifies hierarchically composed systems, the challenges in adopting the approach was not apparent until we applied it “at scale” over an extended period of time. In [5], we demonstrated a proof of concept analysis on an early version of the GPCA model over a subset of system requirements (18 out of the 86 system requirements). In an effort to expand the application of our approach to the entire software of the GPCA, whose complexity and size was substantial, we formalized and verified all its requirements

hierarchically. Among the 86 informally specified system requirements, we identified 55 requirements to be allocated (or flown down) to the software and formalized them (rest of the system requirements exclusively relied on hardware components). Since the software component was further decomposed into 8 sub-components to cope with complexity, we again flowed down requirements to its sub-components that resulted in 102 sub-component requirements. The artifacts developed in this effort is publicly available at [18]. In both the initial effort and the subsequent effort of expanding the scope of GPCA verification, bringing on new team members, and validating our approach, we discovered a number of challenges.

III. FORMAL REQUIREMENTS ANALYSIS CHALLENGES AND MITIGATION

In this section, we elaborate on three challenges that, we believe, have the potential to undermine the rigor and usefulness of formal techniques yet have not been brought to light in any existing literature. In the sequel we explain our approach to addressing them.

A. Challenge 1: Requirements Refinement

When systems are decomposed into components, deriving the component requirements from the system requirements is a challenging task. While formal tools help to automatically verify the sufficiency of component requirements to guarantee the system level requirement, deriving and formalizing those component requirements has been a manual activity. Unfortunately, it has been possible to derive and formalize a set of incorrect and/or infeasible component requirements that the formal tools will verify successfully [19].

In an effort to provide guidance to engineers performing and formally verifying the system decomposition, Miller et. al [8] propose that most system requirements can be recaptured into identical software requirements by slightly modifying the scope of the requirement. Their approach is based on the famous four variable model [20] that formally captures the high level artifacts of a typical process control system, the requirements (*REQ*), the sensor functions(*IN*), controller functions(*SOF*), actuator functions (*OUT*) and their environment (*NAT*). The main contribution of the four variable model is mathematically defining the artifacts, their scope (the four variables - **monitored**, **controlled**, **input** and **output** used to express the artifacts) and their inter-relationships required to reason about their correctness. Miller et. al extended the four variable model, as shown in Figure 1. They recreate virtual versions of the variables *mon'* and *con'* that differ from the original *mon* and *con* in terms of value and timing introduced when sensing and setting the input and output variables. Using these virtual variables, they “stretch” the *SOF* (software) relationship into *IN'*, *REQ'*, and *OUT'*. The *IN'* and *OUT'* represents the specification of hardware drivers that were previously part of the *SOF*. With this change, they propose a mere recapture of each function in *REQ* to *REQ'* using corresponding variables. They assert that this makes the tracing of the requirements *REQ* to the software (*REQ'*) direct

and straightforward. While this approach superficially seems indisputable and makes the task of decomposition simpler, in our experience, we found that this approach could result in inaccurately specified and verified component requirements.

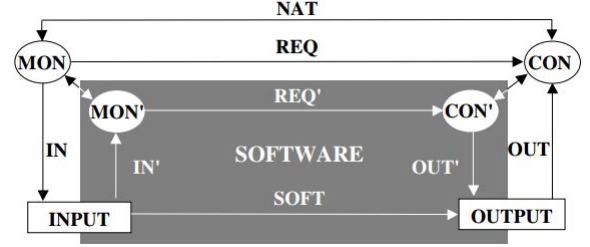


Fig. 1: Extension of Four Variable Model

The overall system requirements especially for complex control systems such as GPCA are typically captured in such a way to accommodate certain degree of inaccuracies and imperfections, that is they are inherently relational. For example, consider a system level requirement:

“In basal mode, the system shall infuse the drug at a flow rate within $basal_flow_rate \pm t\%$ tolerance”

The tolerance (*t*) in this requirement was included to accommodate inaccuracies of the physical components that is going to be used in the GPCA. When we wanted to identify software requirements of the GPCA, we followed the above guidance and formalized the software requirements in a way that mirrors the system requirement but in terms of inputs and outputs of the software, such as:

```
(Mode = basal) ⇒
  FlowRate ≤ (1 + t/100) * (BasalFr) and
  FlowRate ≥ (1 - t/100) * (BasalFr)
```

While the above formulation appears to be correct as per the guidance, the tolerance part was not intended to be allocated to the software. When we verified requirement it was successfully verified since the software indeed satisfies the requirement, but in an unintended fashion. In reality, the software’s output is deterministic and there was no need for the tolerance, hence the consequent in the formalization was supposed to be (*FlowRate = BasalFr*). That is, mathematically the requirement of the software is a function as opposed to the system level requirement that is a relation.

While superficially this doesn’t appear to be a problem, an indepth analysis reveals that this mere recapture presumes an implicit assumption that the physical components of the GPCA are perfect. This could lead to a situation in which if the software is compositionally verified in its environment (with physical components) it would fail. The way we formalized the software requirement is not only an over approximation of the capabilities of the software but also masks the other component requirements. While we were able to formally verify if the sub-components satisfies the software requirements, the correctness of the requirements allocated to the software was

not formally establishable, since they were originally derived from informal system requirement. Unfortunately, the error in the requirements allocation described was not apparently visible until we started documenting an argument about how software requirements realizes the system level requirements.

1) *Solution : Satisfaction Argument*: To identify issues with the requirements allocation we propose documenting *satisfaction argument* between each system (or component) requirement and its allocated component (or sub-component) requirements at adjacent levels in the hierarchy, that captures the relationship between them as well the assumptions that are necessary to establish that relationship. This sort of argument, that is inspired from Hammond’s work [21], helped us understand the role of each component in satisfying the system requirement for GPCA. In fact, by documenting the argument we were able to validate the assumptions made for each requirement that consequently helped identify 8 incorrectly formalized component requirements of the GPCA software. While this documentation is straightforward when it is done at the time of requirements allocation, establishing the traceability between the requirements and its assumptions after all requirements are allocated might be laborious. To address that concern, we are currently working towards building an automated traceability capability in the AGREE tool. This capability would automatically identify for every system requirement (parent level) the set of child component requirements that contributed to satisfying it. The task left for the developer is analyze document the traceability and validate it. While we acknowledge that, if one could formally models and specifies requirements of every component of the system, we could use the tools to analyze the composition and identify the requirement errors. However, in practice, whenever informality is involved we recommend documenting a satisfaction argument for every requirement to validate the requirements flow down and assumptions.

B. Challenge 3: Cursory Knowledge of Tools

Although we used sophisticated tools with advanced capabilities such as consistency checks, cursory knowledge about how the tools perform such checks lead us to successfully verify inconsistent requirements. The tool we used – AGREE – had capability to check consistency of requirements in addition to verifying them that detects logical contradictions among requirements. The intent of performing such checks is to identify if the verification was trivially successful due to presence of inconsistent requirements. While the tool declared that all the GPCA requirements are consistent, our tool expert during the manually inspection phase found that there were several self-inconsistent requirements. To our surprise, those inconsistencies were not reported by the tool. This was primarily because we did not have full knowledge of the tool settings associated with that check.

Let us illustrate the problem with an concrete example. To formalize certain requirements for GPCA we had to specify counters to keep track of duration of internal conditions or occurrence of certain actions. For example, consider a

requirement to notify the clinician when the patient requests more than a certain number of boluses. To formalize this requirement we had to count the number of requests received (`PatientRequest`) by the system. Hence, we declared an integer variable (`boluscnt`), whose initial value was 0 and then gets incremented whenever a `PatientRequest` is received by the system, as shown below.

```
boluscnt:int = 0 -> boluscnt + PatientRequest
```

The formal language used to capture the requirements in AGREE is based on Property Specification Language (PSL) [22] and defines a Lustre language [23] “flavor” for the PSL expressions. Lustre is a synchronous dataflow language that describes the behavior of a system through a set of equations, and it can be viewed as a textual analogue to Simulink block diagrams. In this notation, it is possible to define constants, local variables and reusable fragments of temporal logic (called properties). Additionally, we can describe stateful relationships between variables using the ‘pre’ expression, which provides the value of a variable from the previous step of execution of the system. In this formalization, “->” is used to capture the sequence of value computations for a variable. For example “0 -> 5” means the value is 0 in the initial execution (or time) step and the value is 5 in all subsequent execution steps.

While the tool successfully verified the requirement involving the `boluscnt` variable, during manual inspection our tool expert pointed out this statement was internally inconsistent. The inconsistency was due to the usage of `boluscnt` in both the right and left side of the equation. According to the tool, the value for `boluscnt` always refers to its value in the current time step, whereas we actually intended to refer to the value in the previous time step in the right side of the expression. The incorrect formulation had caused a circular dependency in assigning values for `boluscnt` variable, that made this expression and the requirement involving this variable inconsistent and trivially satisfied. In the GPCA, we identified 20 such inconsistent formalizations in both system and component levels. To fix this issue, we used AGREE’s operator “*pre*”, to refer to the value of a variable in the previous step as shown below.

```
boluscnt:int = 0 -> pre(boluscnt) + PatientRequest
```

While adding “*pre*” fixed this specific type of inconsistency, our concern was that the tool’s consistency checker did not identify this problem. After discussing with the tool developers, we found that the tool was set up by default to check for consistency only in the initial time step (first step of execution). This was not a bug in the tool, rather an intentional default setting to optimize the performance of the tool. According to the tool developers, most inconsistencies can be found in the first step and this was an exceptional condition. Since the `boluscnt` equation was inconsistent only after the

initial step (its initial value was 0), the consistency checker did not report the problem.

In our opinion, this situation is not limited to this specific example or tool but generalizable for all model checking tools and techniques. The root cause of this problem is not just our cursory knowledge of the tools, but also the lack of adequate details of the task displayed by the tool.

1) *Solution: Understanding Tool Settings:* We believe that the root cause of the AGREE tool not being able to identify the consistency issues in the formalized requirements was because the tool had an incorrect default setting, but the fact that we were not aware of that setting and it was not apparent in the results. Hence, to address the issue from the tool side we requested the tool developers to provide elaborate messages of certain key configurations when using the features, that play a key role in determining their correctness. After that change, AGREE now displays the depth of consistency check along with the results. While such a change may not be possible with every tool and feature we use, we strongly recommend the engineers to delve deep into the details and configuration of tool to precisely understand the results, especially when there is limited information provided by the tool along with the results.

C. Challenge 3: Matching Tool Boundaries

While it is ideal to use a tool/technique to verify the entire system's requirements, in practice it is inevitable to avoid the use of multiple tools for scalability concerns and different verification needs. One of the well known problems associated with using multiple tools is matching the semantic differences between them. In an earlier effort, in which we semi-formally mapped abstraction differences between infusion pump requirements formalized and captured as timed automata and AGREE notation, we identified semantic errors in the mapping, in particular differences in the way time is handled in the notations. We fixed those issues by defining a formal semantics [24] to rigorously map the models and their requirements. However, we found that even when tools with "same" semantics were used rigorously maintaining the relationship between them in the long run is challenge.

In GPCA, we used a combination of two tools to cope with scalability of verification. We used AGREE to model the system architecture and verify the decomposition of system requirements into component requirements. Subsequently, we also developed detailed behavioral models for each component in Simulink and verified if the component requirements verified in AGREE are indeed satisfied by the respective component's behavioral model using Simulink Design Verifier (SDV). The advantage of this tool combination in addition to being scalable is that the semantics of the notation used in AGREE and SDV (Embedded Matlab) to specify requirements are comparable. Hence, we presumed that the translation of requirements between AGREE and Simulink notations can be done in an error free manner.

While a majority of the requirements were correctly recaptured between the notations some requirements were incor-

rectly recaptured that, unfortunately, was not easy to detect. We identified two such issues - transcription errors and requirement management issues. The transcription errors occurred in the process of manually recapturing requirements. When recapturing some requirements we unknowingly changed the operators. For example, consider the following capture of requirements between AGREE and Simulink:

```
Properties in Embedded Matlab:
-----
sldv.prove(
    implies((SystemOn and AirInLine),
            (FlowRate <= MedFlow))
sldv.prove(
    implies((SystemOn and EmptyReservoir),
            (FlowRate <= LowFlow))

Property in AGREE:
-----
guarantee "Property1":
    (SystemOn && AirInLine) => (FlowRate = MedFlow)

guarantee "Property2":
    (SystemOn && EmptyReservoir) => (FlowRate = LowFlow)
```

It was not possible to dismiss this error in the operator as a mere typo, since it demonstrates the potential for occurrence of such issues. Unfortunately, this error was not visible until we checked for feasibility using a recent enhancement of AGREE's tool called *realizability*. Intuitively, realizability checks if there exists an output of the system for every input that satisfies the requirements. In the above case, the realizability check identified a counterexample in which both AirInLine and EmptyReservoir¹. While this feature was not intended to identify the recapture errors, coincidentally we were able to identify this issue. After the found this errors, on manual inspection we identified 6 such transcription issues and corrected them.

Another issue, that we believe is more common, is maintaining the synchrony between the requirements in both tools. After we proved requirements in AGREE, when we recaptured it for verification using SDV, at times we had to slightly change the way requirements were formalized, such that it is verifiable in the behavioral Simulink. However, such changes were sometimes missed to be changed and reverified in AGREE - another human error that caused mismatch between the requirements. Although we were diligent in recapturing the requirements between the tools, human errors and negligence was unavoidable. For the GPCA, we did additional manual inspections to verify the synchrony. However, in our opinion, a lack of rigours process and/or automation to translate and check for synchrony is the root cause of this problem.

1) *Solution: Automated Translators and Manual Inspections:* To address the transcription errors for GPCA, we first strengthened the process of verifying the translation using code inspection by a different developers. This inspection was effective in identifying errors, but was time consuming. Further, performing inspections to maintain the synchrony

¹AirInLine indicates a hazard in which air bubble(s) are detected in the infusion tubing and EmptyReservoir is a hazard that indicates that the drug reservoir does not have sufficient drug to infuse

between the requirements in both the tools in the long run was challenging. This challenge became the motivation for the tool developers to enhance their tool by an automated translator that recaptures properties from AGREE tool notation to Embedded Matlab notation and vice-versa. While we are not actively involved in the development of that automation, we are currently verifying and validating their work using the GPCA's manual translation. While code inspections were effective, we strongly recommend automated translators since they significantly reduced the overall time.

IV. DISCUSSION AND CONCLUSION

In this paper, we elaborated the challenges that we encountered and our approaches to successfully use formal methods for engineering requirements of a complex medical device system. There were a number of useful lessons learned from this effort, especially when applying the techniques "at scale". Based on our experience, we assert that to realize the benefits of using formal methods to verify requirements we need to adapt both the requirements engineering activities to support formal methods as well as the formal techniques to recognize common requirement verification concerns.

In practice, mapping the informal requirements to formal statements, especially at different levels of system abstraction can not be completely avoided while developing systems. In such cases, establishing the fidelity between them is a challenge. Our recommendation to capture the *satisfaction argument* for every requirement, forces one to document how the component requirements and assumptions (if any) contributed to satisfying the system requirements. We believe that, this argument helps validate the correctness the requirements flown down to components as well as identify unrealistic assumptions with respect to every requirement.

While improving the quality of formalized requirements is the goal for the requirements engineers, we suggest that the developers of formal tools should identify "hazardous" use and fallibilities of their tools and mitigate them by either providing guidance or making them apparent in their tool's user interface. While we were able to request the developers of AGREE tool to enhance the capabilities of AGREE, we recommend the developers of formal tools to proactively build such capabilities.

In sum, we believe that the lessons we learned in this effort was not limited to the infusion pump, but are broadly applicable to practitioners working on related applications and that our experience serves informative.

REFERENCES

- [1] S. P. Miller, A. C. Tribble, and M. P. E. Heimdahl, "Proving the shalls," in *Proceedings of the Int'l Symposium of Formal Methods Europe (FME 2003)*, ser. Lecture Notes in Computer Science, K. Araki, S. Gnesi, and D. Mandrioli, Eds., vol. 2805. Pisa, Italy: Springer, September 2003, pp. 75–93.
- [2] M. W. Whalen, D. D. Cofer, S. P. Miller, B. H. Krogh, and W. Storm, "Integration of formal analysis into a model-based software development process," in *FMICS*, ser. Lecture Notes in Computer Science, S. Leue and P. Merino, Eds., vol. 4916. Springer, 2007, pp. 68–84.
- [3] R. Kneuper, "Limits of formal methods," *Formal Aspects of Computing*, vol. 9, no. 4, pp. 379–394, 1997.

- [4] A. Hall, "Seven myths of formal methods," *Software, IEEE*, vol. 7, no. 5, pp. 11–19, 1990.
- [5] A. Murugesan, M. W. Whalen, S. Rayadurgam, and M. P. Heimdahl, "Compositional verification of a medical device system," in *ACM Int'l Conf. on High Integrity Language Technology (HILT) 2013*. ACM, November 2013.
- [6] A. Murugesan, M. P. Heimdahl, M. W. Whalen, S. Rayadurgam, J. Komp, L. Duan, B.-G. Kim, O. Sokolsky, and I. Lee, "From requirements to code: Model based development of a medical cyber physical system," *Fourth International Symposium on Software Engineering in Healthcare workshop (SEHC 2014)*, 2014.
- [7] A. Murugesan, O. Sokolsky, S. Rayadurgam, M. Whalen, M. Heimdahl, and I. Lee, "Linking abstract analysis to concrete design: A hierarchical approach to verify medical cps safety," *2014 ACM/IEEE International Conference on Cyber-Physical Systems (ICCPs)*, vol. 0, pp. 139–150, 2014.
- [8] J. M. Thompson, M. P. Heimdahl, and S. P. Miller, "Specification-based prototyping for embedded systems," in *ACM/SIGSOFT Symposium on the Foundations of Software Engineering*. Springer, 1999, pp. 163–179.
- [9] R. D. Jeffords, C. L. Heitmeyer, M. M. Archer, and E. I. Leonard, "Model-based construction and verification of critical systems using composition and partial refinement," *Formal Methods in System Design*, vol. 37, no. 2-3, pp. 265–294, 2010.
- [10] M. Kauppinen, J. Savolainen, and T. Mannisto, "RE theory meets software practice: Lessons from the software development trenches," in *Requirements Engineering Conference, 2007. RE'07. 15th IEEE International*. IEEE, 2007, pp. 265–268.
- [11] D. L. Lempia and S. P. Miller, "Requirements engineering management handbook," *National Technical Information Service (NTIS)*, 2009.
- [12] FDA, "White Paper: Infusion Pump Improvement Initiative," April 2010.
- [13] A. Mavin, P. Wilkinson, A. Harwood, and M. Novak, "Easy approach to requirements syntax (ears)," in *17th IEEE Int'l Requirements Engineering Conf.* IEEE, 2009, pp. 317–322.
- [14] "IEEE recommended practice for software requirements specifications," *IEEE Std 830-1998*, pp. 1–40, Oct 1998.
- [15] P. H. Feiler, B. A. Lewis, and S. Vestal, "The sae architecture analysis & design language (aadl) a standard for engineering performance critical systems," in *2006 IEEE Conference on Computer Aided Control System Design, 2006 IEEE International Conference on Control Applications, 2006 IEEE International Symposium on Intelligent Control*. IEEE, 2006, pp. 1206–1211.
- [16] "The MathWorks Inc." <http://www.mathworks.com>, 2015.
- [17] D. D. Cofer, A. Gacek, S. P. Miller, M. W. Whalen, B. LaValley, and L. Sha, "Compositional verification of architectural models," in *Proceedings of the NASA Formal Methods Symposium*, vol. 7226, April 2012, pp. 126–140.
- [18] "Generic patient controlled analgesia infusion pump project - <http://crisys.cs.umn.edu/gpca.shtml>."
- [19] A. Gacek, A. Katis, M. W. Whalen, J. Backes, and D. Cofer, "Towards realizability checking of contracts using theories," in *Proceedings of the NASA Formal Methods Symposium*. Springer, May 2015, pp. 173–187.
- [20] D. L. Parnas and J. Madey, "Functional documentation for computer systems engineering (volume 2)," McMaster University, Hamilton, Ontario, Tech. Rep. CRL 237, September 1991.
- [21] J. Hammond, R. Rawlings, and A. Hall, "Will it work? [requirements engineering]," in *Proceedings of the IEEE Int'l Symposium on Requirements Engineering*, 2001, pp. 102–109.
- [22] *IEEE Std. Property Specification Language (PSL)*. IEEE, 2005.
- [23] P. Caspi, D. Pilaud, N. Halbwachs, and J. A. Plaice, "LUSTRE: A declarative language for programming synchronous systems," in *ACM Symp. Principles Program. Lang. (POPL)*, Munich, Germany, 1987, pp. 178–188.
- [24] M. W. Whalen, S. Rayadurgam, E. Ghassabani, A. Murugesan, O. Sokolsky, M. P. Heimdahl, and I. Lee, "Hierarchical multi-formalism proofs of cyber-physical systems," in *Formal Methods and Models for Codesign (MEMOCODE)*, 2015 ACM/IEEE International Conference on. IEEE, 2015, pp. 90–95.